GlareTracker

Code Architecture & Technical Documentation

Version 1.0.0

Project: Windshield Glare Tracking System

Purpose: Track driver eye position via webcam, compute headlight ray-windshield intersections, visualize in real-time

Stack: Python 3.11 | PyQt5 | OpenCV | MediaPipe | PyQtGraph

1. System Overview

GlareTracker is a real-time desktop application that simulates and visualizes how headlight glare from an oncoming vehicle strikes a driver's windshield. The system:

- Captures the driver's face via laptop webcam
- Tracks the iris position using MediaPipe Face Mesh (468+ landmarks)
- Converts pixel coordinates to real-world centimetre positions
- Computes where the eye-to-light ray intersects the windshield plane
- Visualizes everything in a 5-panel dashboard (camera feed, 3D scene, 2D windshield, telemetry readout, controls)

2. Physical Setup & Coordinate System

Coordinate Axes

Axis	Direction	Example
X	Lateral (left/right)	X=-10 = driver looking left
Y	Depth (front/back)	Y=0 windshield, Y=-50 driver
Z	Vertical (height)	Z=0 bottom, Z=19.5 top

Key convention: The windshield sits in the vertical X-Z plane at Y=0. The driver is at negative Y (behind it). The light source is at positive Y (in front).

3. Project File Structure

```
Glare suppression system_test2/
+-- GlareTracker.pyw            # Silent launcher (no console)
+-- run.py                      # Command-line launcher
+-- glare_tracker/              # MAIN APPLICATION PACKAGE
|   +-- __init__.py             #   Package metadata
|   +-- __main__.py             #   python -m support
|   +-- app.py                  #   QApplication setup
|   +-- config.py               #   TrackingConfig dataclass
|   +-- geometry.py             #   Intersection math
|   +-- styles.py               #   Dark theme stylesheet
|   +-- camera.py               #   Webcam capture thread
|   +-- tracking.py             #   TrackingController
|   +-- window.py               #   MainWindow (layout)
|   +-- widgets/                #   UI components
|       +-- video.py            #     Camera feed
|       +-- windshield_2d.py    #     2D windshield view
|       +-- scene_3d.py         #     3D OpenGL scene
|       +-- readout.py          #     Telemetry readout
|       +-- controls.py         #     User controls
+-- assets/                     # Icon files
+-- docs/                       # Documentation
```

```
  +-- scripts/                    # Build scripts
  +-- dist/                       # Standalone executables
```

4. Software Architecture

Design Principles

- Separation of Concerns: Each module has a single responsibility
- MVC-inspired: Controls (Input) -> TrackingController (Logic) -> Widgets (Output)
- Non-blocking I/O: Camera runs in QThread; GUI thread only processes signals
- In-place updates: 3D objects created once, updated via setData() for 60+ FPS

Class Overview

Class	Parent	Module	Role
TrackingConfig	dataclass	config.py	Immutable config snapshot
VideoCaptureThread	QThread	camera.py	Camera I/O + MediaPipe
TrackingController	object	tracking.py	All business logic
MainWindow	QMainWindow	window.py	Layout + signal wiring
VideoWidget	QLabel	widgets/video.py	Camera feed display
Windshield2DWidget	QWidget	widgets/windshield_2d.py	2D glare view
Scene3DWidget	QWidget	widgets/scene_3d.py	3D OpenGL scene
ReadoutPanel	QGroupBox	widgets/readout.py	Telemetry text
ControlsPanel	QGroupBox	widgets/controls.py	User inputs

5. Module-by-Module Breakdown

5.1 config.py - Configuration

Contains the TrackingConfig frozen dataclass. An immutable snapshot of tracking parameters passed from the GUI thread to the camera thread. Being frozen makes it inherently thread-safe.

Field	Type	Default	Description
screen_w_cm	float	34.5	Windshield width
screen_h_cm	float	19.5	Windshield height
eye_dist_cm	float	50.0	Driver distance
camera_x_cm	float	0.0	Camera X offset
camera_y_cm	float	0.0	Camera Y offset
camera_z_cm	float	30.0	Camera height

5.2 geometry.py - Intersection Math

Two pure functions with NO UI dependencies (easy to unit test):

compute_intersection(eye_cm, light_cm)

Computes where the ray from the driver's eye toward the light source crosses the windshield plane at Y=0 using parametric line-plane intersection:

P(t) = eye + t * (light - eye)

Solve: eye.y + t * (light.y - eye.y) = 0

t = -eye.y / (light.y - eye.y)

The t parameter is clamped to [0,1] so only intersections between the eye and light are reported.

pixel_to_cm(...)

Converts pixel-space iris midpoint from MediaPipe to real-world centimetres, mapping camera frame dimensions to physical screen dimensions.

5.3 camera.py - Webcam Capture Thread

Runs in a background QThread so the GUI never freezes (~33ms per frame).

Decision	Rationale
cv2.CAP_DSHOW	DirectShow gives instant startup on Windows
QImage.copy()	Qt must own buffer to prevent crash
BGR->RGB in worker	Offloads CPU from GUI thread
Iris landmarks 468 & 473	True iris centre landmarks
try/except/finally	Ensures camera always released

Signals Emitted

Signal	Data	When
frame_ready(QImage)	RGB frame + overlay	Every frame (~30/sec)
eye_detected(x,y,z)	Eye position in cm	Face found
eye_lost()	None	No face detected

5.4 tracking.py - Tracking Controller

The most important architectural extraction. Previously all logic lived inside MainWindow (violating Single Responsibility). TrackingController is a plain Python class that receives raw eye coordinates, applies calibration, computes intersections, updates all UI panels, manages movement simulation, and counts FPS.

Method	What it does
on_eye_detected(x,y,z)	Applies calibration, triggers process_tracking()
on_eye_lost()	Clears all visual indicators
process_tracking(eye)	Computes intersections, updates 3D/2D/readout
calibrate()	Sets current eye as centre reference
start_movement()	Begins light animation
advance_movement()	Moves lights one tick (50ms interval)
on_params_changed()	Reacts to control panel changes
update_fps()	Calculates frame rate from timestamps

5.5 window.py - Main Window

After refactoring, MainWindow is a thin shell that only handles: widget creation, layout (nested QSplitters), signal wiring, stream toggle, and responsive resize. Zero tracking logic, zero geometry, zero calibration math.

5.6 widgets/ - UI Components

Widget	Lines	Description
VideoWidget	~35	QLabel displaying camera frames with aspect ratio
Windshield2DWidget	~140	Custom QPainter 2D front view with glowing glare s
Scene3DWidget	~340	PyQtGraph GLViewWidget managing all 3D objects
ReadoutPanel	~90	Monospace telemetry with dynamic intersection list
ControlsPanel	~140	All user inputs organized in collapsible groups

6. Data Flow & Signal Architecture

Eye Tracking Pipeline

```
Camera sensor -> VideoCaptureThread -> TrackingController -> Widgets

  1. cv2.read()            5. Apply calibration
  2. MediaPipe FaceMesh    6. Compute intersections
  3. Detect iris           7. Update 3D/2D/readout
  4. pixel_to_cm()         8. Emit frame to VideoWidget
```

Qt Signals Used

Signal	From	To
frame_ready	VideoCaptureThread	VideoWidget.update_frame
eye_detected	VideoCaptureThread	TrackingController.on_eye_detected
eye_lost	VideoCaptureThread	TrackingController.on_eye_lost
valueChanged (x9)	QDoubleSpinBox	MainWindow._on_params_changed
clicked	btn_stream	MainWindow._toggle_stream
clicked	btn_calibrate	TrackingController.calibrate
timeout (1s)	QTimer	TrackingController.update_fps
timeout (50ms)	QTimer	TrackingController.advance_movement

7. Threading Model

```
GUI THREAD (main)              WORKER THREAD
  MainWindow                     VideoCaptureThread
  TrackingController             cv2.VideoCapture
  All Widgets                    MediaPipe FaceMesh
  QTimers                        BGR->RGB conversion
                                 QImage construction
  <-- signals (thread-safe, auto-queued) --
```

Thread safety: Qt signals across threads are auto-queued. TrackingConfig is frozen/immutable, safe to read from worker thread.

8. Feature Details

Vehicle Light Modes

Category	Lights	Offsets
Motorcycle	1	(0, 0, 0)
Four-Wheeler/Car	2	(-8,0,0) and (+8,0,0)

Extreme Vehicle	4	(+-10,0,-4) and (+-10,0,+4)

Movement Simulation

Pattern	Description
Coming straight	Approaches from Y+240 with lateral sine wave
Coming at opposite side	Sweeps laterally across 180cm (X axis)
Long distance	Approaches from Y+700 to near

Animation: 50ms intervals (20 ticks/sec), phase +0.0125/tick, full cycle = 4 seconds.

9. Build & Distribution

Running from Source

```
pip install -r requirements.txt
python run.py    # or: python -m glare_tracker
                 # or: double-click GlareTracker.pyw
```

Building Standalone

```
./scripts/build_exe.ps1
# Output: dist/GlareTracker.exe (~270 MB)
```

Dependencies

Package	Purpose
PyQt5 >= 5.15	GUI framework
opencv-python >= 4.8	Webcam capture & image processing
mediapipe >= 0.10	Face mesh & iris detection
numpy >= 1.24	Array operations
pyqtgraph >= 0.13	GPU-accelerated 3D visualization
PyOpenGL >= 3.1	OpenGL bindings for pyqtgraph

10. Glossary

Term	Definition
Intersection	Point where eye-to-light ray crosses windshield (Y=0)
In-bounds	Intersection within windshield rectangle
Calibration offset	Difference between actual and centre position
Baseline	First captured eye position (auto-set)
Movement phase	0.0-1.0 progress through animation cycle
GLTextItem	PyQtGraph 3D billboard text label
Frozen dataclass	Immutable Python dataclass (thread-safe)
DirectShow	Windows camera API for fast startup